DuocUC® ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Sistema de Auditoría para Servidores Privados de Ragnarok Online

---

Caso de análisis semestral para Taller de Base de Datos
(Evaluación 2)

Integrantes:
José Calderón
Sebastian Apablaza

Taller de Base de Datos (BDY1103-002D) Prof. Pablo Salamanca M.

Índice

Índice **2**
Introducción **3**
Descripción del Proyecto 3
Alcance 3
Tecnologías utilizadas 4
Primera propuesta: desarrollo de Bloques PL/SQL Anónimos **4**
Desarrollo de procedimientos y funciones **5**
Desarrollo de paquetes **6**
Desarrollo de triggers **7**
Conclusión **7**
Referencias **8**
Anexos **8**

Introducción

Descripción del Proyecto

Ragnarok Online es un videojuego MMORPG coreano de culto cuyo ecosistema de servidores privados opera bajo **rAthena**, un emulador de código abierto hecho por la comunidad, distribuido en tres servicios principales: *login* (autenticación), *char* (gestión de personajes) y *map* (lógica del juego en tiempo real). Por razones de compatibilidad histórica y priorización de rendimiento síncrono, la base de datos operacional utiliza un esquema desnormalizado y motores de almacenamiento legados (como MyISAM). Esta arquitectura carece de claves foráneas nativas, aplica bloqueos a nivel de tabla completa que por ende limita el cumplimiento estricto de las propiedades ACID. Como resultado, la base de datos es vulnerable a inconsistencias relacionales, generación o duplicación ilegítima de ítems y vacíos de trazabilidad originados por la coexistencia de tablas SQL con catálogos en archivos estáticos (YAML).

Para mitigar estas grietas de integridad, el proyecto implementa una solución de Sistema de Auditoría e inteligencia de datos sobre una base de datos Oracle moderna. Mediante la técnica Extract, Load, Transform (ELT) se procesan las tablas más apropiadas mediante PL/SQL para consolidar métricas, evaluar patrones atípicos de comportamiento en los jugadores y detectar anomalías económicas o comportamientos sospechosos.

Estas auditorías permiten identificar ítems que no se encuentran registrados en los catálogos oficiales, antecedentes de generación de objetos mediantes comandos Game Master (GM ->@item <id>), posibles multicuentas asociadas a direcciones IP compartidas y personajes cuyos puntos de habilidad superan los valores esperados según las reglas evaluadas. Los hallazgos se almacena en reportes y bitácoras que facilitan su revisión por parte de los administradores, sin intervenir directamente en los datos originales ni determinar automáticamente la existencia de una infracción.

Para cumplir este objetivo se desarrollaron varios objetos PL/SQL, procedimientos que ejecutan cada auditoría, funciones que encapsulan las reglas de negocio (validar una IP por ejemplo), paquetes que agrupan ambos objetos anteriores y se dividen por responsabilidad (algunos para las herramientas de auditoria, otro para instalación de tablas, logs, etc) y triggers que protegen modificaciones de algunas tablas y automatizan la creación de permisos y sinónimos. Adicionalmente, una nueva funcionalidad es la integración de herramientas para agentes de Inteligencia Artificial mediante el servidor MCP (Model Context Protocol) que ofrece Oracle en sus Bases de Datos Autónomas.

Alcance

rAthena cuenta con más de 70 tablas en su base de datos para los tres servidores, además de 40 archivos yaml que almacenan definiciones básicas del juego, por rendimiento y legibilidad. Para este proyecto y fase inicial se seleccionaron las tablas y archivos yaml más aptos para realizar auditoria y análisis, los cuales son:

- atcommandlog: logs de comandos hechos por game masters (administradores del juego).
- char: información de un personaje, una cuenta puede tener varios personajes.
- inventory: inventorio de un personaje.
- ipbanlist: lista de IPs bloqueadas del servidor.
- item_db_equip: convertido desde yaml, contiene información de items equipables, como armaduras o armas.
- item_db_etc: convertido desde yaml, contiene información de items miscelaneos.
- item_db_usable: convertido desde yaml, contiene información de items consumibles, como pociones, comida.
- login: información de la cuenta, contiene nombre, contraseña, ultima IP de la sesión, etc.
- picklog: registro de obtención de ítems por parte de personajes (char).
- quest: registro de las misiones de un personaje.
- skill: datos de habilidades que tiene un personaje y su nivel.
- skill_db_data: descripción de las habilidades.

En Anexos, Figura 1 se encuentra el diseño lógico de las tablas con más detalle.

Tecnologías utilizadas

Para la realización de este proyecto se utilizó una base de datos externa al sistema de rAthena, en especifico se trata de una Base de datos con IA Autónoma de Oracle versión 26ai, con una carga de trabajo Lakehouse apta para OLAP, lo que significa que el motor está optimizado para consultas complejas a diferencia de una carga transaccional (OLTP), que prioriza la eficiencia en transacciones rápidas y sencillas.

En este sentido, empleamos PL/SQL, que extiende el alcance del SQL tradicional hacia la lógica de un lenguaje de programación propiamente tal, con beneficios que profundizaremos en las siguientes secciones. Los bloques PL/SQL y configuración de los esquemas se realizaron con Oracle SQL Developer.

Para diagramar el esquema a utilizar, se utilizó Oracle Data Modeler.

También se utilizó el Servidor MCP integrado de Oracle, que mediante el paquete DBMS_CLOUD_AI_AGENT se pueden exponer funciones PL/SQL como herramientas invocables por un agente de IA.

Primera propuesta: desarrollo de Bloques PL/SQL Anónimos

Antes de implementar los objetos de auditoría, estos se esbozaron previamente como bloques anónimos de PL/SQL (se pueden ver en Anexos), con el uso de variables tipo RECORD y VARRAY, utilizados para recorrer cursores explícitos, complejos y

parametrizados, guardando los resultados en una tabla y manejando errores mediante excepciones, algunas predefinidas como COLLECTION_IS_NULL, OTHERS y definidas por usuario cuando se rompía una regla de negocio. Esta modalidad rápidamente mostró sus limitaciones, partiendo con el hecho de que los bloques no se guardan, deben ser escritos y ejecutados en cada utilización, también tenian limitada reutilización, un bloque no podía usar una función de otro, por ejemplo; y las inserciones a la tabla de excepciones se incluyen en la transacción principal. Estas y más limitaciones se resuelven con la elaboración de objetos de base de datos: procedimientos, funciones, paquetes y triggers que veremos a continuación.

Desarrollo de procedimientos y funciones

Tanto los procedimientos como funciones son bloques con nombre que se compilan y se almacenan en la base de datos, lo que permite que se ejecuten repetidamente.

Un procedimiento es un objeto que ejecuta una acción, no retorna un valor por sí mismo (aunque tecnicamente tiene un método alternativo de parámetros OUT) y se invoca mediante EXEC, desde un bloque anónimo, un subprograma, un trigger o un job.

Una función en cambio retorna obligatoriamente un valor con RETURN, tambien puede utilizarse dentro de otro programa PL/SQL y si es pública y no modifica datos, puede utilizarse en sentencias SQL.

Ambos se compilan una vez y quedan almacenados, aceptan parámetros de entrada y estos mismos pueden heredar tipos de filas o columnas con %TYPE y %ROWTYPE. Por defecto se ejecutan con derechos del creador, pero se puede ajustar con AUTHID para usar los permisos del invocador solamente, y sus accesos se controlan con GRANT EXECUTE.

Algunas ventajas de los procedimientos es que pueden encapsular procesos completos y ofrecer control transaccional con SAVEPOINT y ROLLBACK TO. Son reutilizables desde bloques, triggers y paquetes sin reescribir código. También reducen el tráfico de la red porque se ejecutan mediante una sola llamada y directamente en la base de datos. Pero tienen la desventaja de que no son compatibles con SQL normal, si hay un cambio imprevisto en las tablas que utiliza se invalida y obliga su recompilación.

Las funciones tienen la ventaja de poder ser integradas en SQL, por lo que fácilmente pueden probarse de forma aislada. Pero tiene la desventaja de que si se ejecutan por SQL repetidamente para muchos datos afectan el rendimiento debido al cambio de contexto entre motores SQL y PL/SQL, además, si la función ejecuta DML no funcionará mediante SQL, lanzando una excepción, a menos de que el DML se procese mediante una transacción autónoma PRAGMA.

Para este proyecto los procedimientos ejecutan acciones del sistema, es decir, la auditoría sobre las tablas y escriben los resultados en sus respectivas tablas, por ejemplo sp_items_fantasma y sp_items_gm, que utilizan SAVEPOINT y ROLLBACK TO para deshacer cambios en un caso de error antes de propagar la excepción y BULK COLLET con FORALL para procesar por lotes operaciones DML, evitando muchos cambios de contexto, lo que no influye mucho en el rendimiento con los datos actuales en el proyecto, pero preparan al sistema para una futura escalabilidad. También se usaron procedimientos para

generar permisos y sinónimos a través de un job, que es una tarea programada. Todos los procedimientos se pueden apreciar en detalle en el Anexo: Objetos PL/SQL.

Por otro lado las funciones ejecutan reglas del negocio, determinan si una IP tiene formato válido (fn_es_ipv4_valida) o cuántas cuentas pertenecen a una IP (fn_cuentas_por_ip) entre otras funcionalidades que se encuentran en el Anexo: Objetos PL/SQL también.

Se destaca además que varias funciones son la interfaz de consulta que el servidor MCP expone a los agentes de IA, todas retornan un resultado en JSON en formato CLOB (Character Large Object), son adaptadas desde la documentación de Oracle para que registren en la tabla log_explorador cada llamada al servidor, y solo consultan datos, no permiten uso de DDL o DML. Cada usuario con rol desarrollador puede integrar estas funciones mediante un procedimiento.

Desarrollo de paquetes

Un paquete es un objeto de esquema, y agrupa diversos componentes como tipos, variables, cursores, excepciones y por supuesto procedimientos y funciones. Se compone de dos partes: una de especificaciones, o la parte pública, que declara (solamente declara) todo lo que pueda ser referenciado externamente mediante paquete.elemento. Y la segunda parte es el cuerpo, que implementa los subprogramas públicos, si hay algún elemento no declarado en el cuerpo, será privado y no podrá accederse externamente.
Al igual que los procedimientos pueden usar AUTHID para definir los privilegios del ejecutor. Con ACCESSIBLE BY se puede restringir que el paquete pueda ser usado sólo por otros paquetes determinados (incluyendo futuros paquetes que aún no existen), también pueden establecerse excepciones públicas y asociarse con un código de Oracle con PRAGMA EXCEPTION_INIT, y con UTL_CALL_STACK se registra el programa padre donde la excepción se propagó.

La gran ventaja de los paquetes es que modularizan las responsabilidades del sistema claramente, pueden ocultar implementación con elementos privados, y agilizan su uso con un GRANT EXECUTE a un solo paquete en vez de repetir esa sentencia para cada función, al modularizar elementos estos pueden ser compartidos, como excepciones y tipos especificados. No obstante algunas desventajas son que el paquete se carga completo aun si solo se usa solo un subprograma, un cambio en la parte de especificación invalida los programas que dependen de esa parte, y requieren mayor diseño / planificación a diferencia de escribir procedimientos sueltos.

El proyecto consta de 5 paquetes, pkg_bitacora tiene las responsabilidad registrar el uso de herramientas del servidor MCP y el registro de excepciones en una tabla centralizada de ro_data, esto último se realiza al ofrecer el procedimiento registrar_excepcion a otros paquetes mediante ACCESIBLE BY. Los procedimientos usan una funcion compartida que extrae variables de SYS_CONTEXT para obtener todos los datos relevantes para trazabilidad.

pkg_auditoria compone las herramientas de auditoria y eran la mayoria de bloques anónimos en la fase previa, ahora comparten la excepción pública e_formato invalido, y las

excepciones de negocio se establecen como privadas, en vez de VARRAY se optó por utilizar tablas anidadas porque no tienen un máximo definido, aportando flexibilidad al flujo.

pkg_instalacion se encarga de crear las tablas resultantes de pkg_auditoria, tienen excepciones, funciones y procedimientos públicos como e_sin_privilegio, sp_crear_tablas y fn_es_tabla_hallazgo.

pkg_explorador contiene las herramientas MCP para agentes de IA que explicamos anteriormente, para facilitar a los desarrolladores nuevos a comprender los datos del juego en caso de no tener un conocimiento previo o profundo previamente.

pkg_permisos se encarga de crear y deshacer permisos y sinonimos de las tablas de análisis de ro_data. A diferencia de los 3 paquetes anteriores, el AUTHID se ejecuta como creador, porque está pensada para usarse solamente por ro_data y no los desarrolladores. verifica que tablas ya hay creadas, diferencia cuales son de auditoria y cuales son del juego (tomando prestada la función fn_es_tabla_hallazgo del paquete de instalación). los procedimientos crean un job para ejecutar las acciones sobre sinónimos y permisos, la razón se explicará en la sección de triggers.

Desarrollo de triggers

Un trigger o disparador es una entidad PL/SQL, almacenada igualmente, que se ejecuta cuando ocurre un evento determinado, a diferencia de los objetos anteriores, no se invocan explícitamente, sino que se habilitan o deshabilitan. El evento suele ser una operación DML sobre una tabla o vista, pero también puede gatillarse con DDL y operaciones de base como LOGON, que ya estarían siendo triggers de sistema, se ejecutan mediante un momento BEFORE, AFTER o INSTEAD OF, y funcionan a nivel de sentencia o fila (con FOR EACH ROW), este último puede acceder a valores con :OLD y :NEW. No pueden ejecutar COMMIT ni ROLLBACK a menos de que sea una transacción autónoma, y tampoco pueden consultar una tabla que está mutando, porque lanza una excepción (ORA-04091).

La ventaja es que se ejecutan siempre, independiente del cliente, como SQL Developer, un script o un cliente MCP, permitiendo automatizar tareas repetitivas y centralizan reglas de negocio en la base de datos en vez de repetirlas en cada aplicación.

Las desventajas son que al ser “invisibles” podrían dificultar el debugging, y si no se manejan bien pueden ralentizar las operaciones, como un trigger de fila para muchas transacciones. Si el trigger es a nivel DDL y falla, bloquearía la creación o eliminación de tablas. Los triggers DML no cubren el TRUNCATE porque es DDL.

En el proyecto hay 4 triggers principales, 2 de ellos protegen las tablas exception_log y log_explorador para que no puedan eliminarse o cambiarse registros con BEFORE UPDATE OR DELETE y lanza excepciones con códigos definidos por usuario ORA-20001 y ORA-20002. Los otros 2 son de sistema y se ejecutan a nivel DDL con AFTER CREATE (o DROP) ON SCHEMA y derivan los GRANT y CREATE SYNONYM a un job, porque una operación DDL hace commit implícito, y ese commit se traspasaría al trigger, pero como se establece anteriormente éstos no pueden hacer commit ni rollback, desencadenando una

excepción, y una transacción autónoma tampoco funcionará porque no vería la tabla creada o generaría un self-deadlock (ORA-04020).

Conclusión

El proyecto partió como un problema concreto: La antigüedad de la base de datos de rAthena no garantiza la integridad de elementos del juego, o sea no cumple con ACID. La primera aproximación se realizó con bloques anónimos, y las soluciones finales optimizan esos bloques en objetos reutilizables, escalables y mantenibles para una correcta auditoría de los datos en servidores de RO, para detectar anomalías que escapan a las herramientas que ofrece el juego sin afectar el entorno de producción mismo del juego, además de ofrecer un entorno potenciado con IA para facilitar la comprensión y análisis responsable de los datos, manteniendo registros de las acciones derivadas de los estudios, asegurando trazabilidad interna.
Algunos puntos de mejora futura sería la conversión de los archivos .yml a JSON en vez de convertirlos a modelo relacional, ya que la base de datos autónoma de Oracle soporta NoSQL sin problemas a pesar de ser un motor relacional en el fondo. También podrían integrarse las herramientas de auditoría como herramientas MCP para que el agente ejecute el análisis, pero eso requiere cerciorarse de disminuir lo más posible brechas de seguridad en el entorno de producción, para no comprometer los datos del servidor.

Referencias

- Repositorio Github de rAthena
- Documentación oficial de PL/SQL
- Instrucciones para usar servidor MCP en ADB de Oracle
- Oracle: Paquete DBMS_CLOUD_AI_AGENT
- Oracle: PL/SQL Subprograms
- Oracle: PL/SQL Triggers
- Oracle: PL/SQL Packages
- Oracle: DBMS_SCHEDULER
- Oracle: UTL_CALL_STACK
- Oracle: SYS_CONTEXT

Anexos

- **Objetos PL/SQL**
- Bloques Anonimos PL/SQL
- Panel de control aRo referencial
- Bloques y código PL/SQL del trabajo

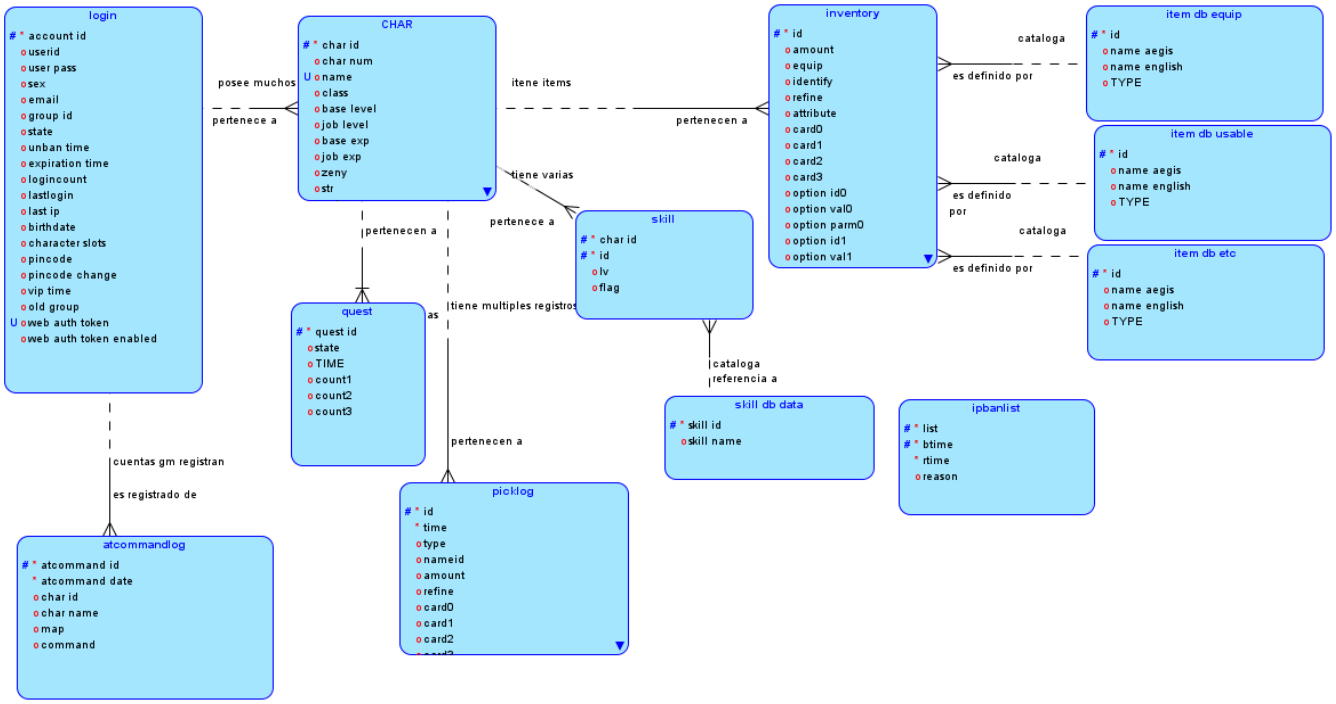


Fig. 1: Modelo lógico de las tablas seleccionadas para auditoria



Fig. 2: Modelo lógico de las tablas de reportes de auditoría